SPECTRE AI — Chrome Extension Build Prompt

> **Purpose**: Build a Chrome Extension that injects Spectre AI's intelligence layer on top of X (Twitter) Smart Cashtags. This is the bridge between X's financial data layer and Spectre's AI-powered research platform.

---

1. WHAT WE'RE BUILDING

A Chrome Extension with **three core views**:

1. **Cashtag Hover Popup** — When a user clicks/hovers any `$TICKER` on X, a Spectre popup appears with price, F&G, AI Pulse, market context, and a CTA to open full research on spectreai.io
2. **Sidebar Panel** — Persistent panel on the right side of X showing AI Pulse, market overview, dominance, trending cashtags, watchlist, and AI Brief
3. **Extension Icon States** — The toolbar icon changes color/badge based on market state (Risk Off = red, Risk On = green, Event Imminent = orange)

We are NOT building: trading execution, wallet connect, social posting bots, or any feature that requires backend infrastructure we don't have yet.

---

2. THE HARD PROBLEM — CASHTAG RESOLUTION

Why This Matters

On X, cashtags are just text strings prefixed with `$`. The same ticker can refer to different tokens across chains. And project names frequently DON'T match their ticker:

Project Name	Cashtag	Why Different
Spectre AI	`$SPECT`	`$SPECTRE` was too long, didn't hyperlink on X
dogwifhat	`$WIF`	Abbreviated meme name
Pepe	`$PEPE`	Matches, but there are 50+ tokens named PEPE across chains
Bitcoin	`$BTC`	Obvious, but which wrapped version? Which chain?
Render	`$RENDER`	Was `$RNDR`, migrated ticker

Resolution Strategy — Three-Tier Lookup

The extension must resolve any cashtag to the correct token using this priority chain:

Tier 1: Known Token Registry (Local, Instant)

Maintain a local JSON registry of well-known tokens. This is the fastest path and handles 90%+ of cashtag interactions on X.

```json
{
  "BTC": {
    "name": "Bitcoin",
    "coingeckoId": "bitcoin",
    "codex": { "address": "0x2260FAC5E5542a773Aa44fBCfeDf7C193bc2C599", "networkId": 1 },
    "icon": "btc",
    "category": "major"
  },
  "ETH": {
    "name": "Ethereum",
    "coingeckoId": "ethereum",
    "codex": { "address": "0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2", "networkId": 1 },
    "icon": "eth",
    "category": "major"
  },
  "SOL": {
    "name": "Solana",
    "coingeckoId": "solana",
    "codex": { "address": "So11111111111111111111111111111111111111112", "networkId": 1399811149 },
    "icon": "sol",
    "category": "major"
  },
  "SPECT": {
    "name": "Spectre AI",
    "codex": { "address": "0x9cf0ed013e67db12ca3af8e7506fe401aa14dad6", "networkId": 1 },
    "icon": "spectre",
    "category": "spectre",
    "aliases": ["SPECTRE"]
  },
  "WIF": {
    "name": "dogwifhat",
    "coingeckoId": "dogwifcoin",
    "codex": { "address": "EKpQGSJtjMFqKZ9KQanSqYXRcF8fBopzLHYxdM65zcjm", "networkId": 1399811149 },
    "icon": "wif",
    "category": "meme"
  }
}
```

This registry should include the top 200 tokens by market cap + any token Spectre's platform actively covers. It should be versioned and updateable without a full extension update (fetch from a remote JSON endpoint on extension startup, cache locally).

Key fields:

- `aliases` — Alternative tickers that should resolve to the same token (e.g., `SPECTRE` → `SPECT`)
- `codex.address` + `codex.networkId` — For direct Codex API lookups (the extension calls our server, not Codex directly)
- `coingeckoId` — For CoinGecko fallback
- `category` — For UI treatment (major, defi, meme, spectre, stablecoin)

Tier 2: Server Search (API Call, ~200ms)

When the cashtag isn't in the local registry, call the existing Spectre server:

```
GET /api/tokens/search?q={ticker}
```

This already exists and uses the resolution chain:

1. Codex `filterTokens` with phrase search
2. Match tier scoring (exact symbol → starts with → contains)
3. Liquidity-weighted ranking (highest liquidity match wins)
4. CoinGecko fallback for majors

The server returns `{ results: [{ token: { symbol, name, address, networkId }, priceUSD, change24, marketCap, liquidity }] }`.

The extension takes the first result with liquidity > $1,000 as the canonical match.

Tier 3: Smart Cashtag Contract Address (Future — When X Launches)

When X enables Smart Cashtags with smart contract addresses, the cashtag itself will contain the exact contract. The extension should detect this format and use it directly:

```
$BTC:0x2260FAC5E5542a773Aa44fBCfeDf7C193bc2C599
```

Parse the contract address from the cashtag, detect the chain from the address format (0x = EVM, base58 = Solana), and call Codex directly.

For now, this is not implemented — but structure the code so it's easy to add when X ships the feature.

Resolution Caching

- **Session cache**: Once a cashtag is resolved, cache the result for the browser session (Map object in background service worker)
- **Persistent cache**: Store the last 500 resolved tokens in `chrome.storage.local` with a 24-hour TTL
- **Cache key**: Uppercase ticker string (e.g., `"BTC"`, `"SPECT"`)
- **Cache invalidation**: Prices refresh every 60 seconds, but the token identity (address, name, networkId) stays cached for 24h

---

3. ARCHITECTURE

```
spectre-chrome-extension/
├── manifest.json              # Manifest V3
├── background/
│   ├── service-worker.js      # Background service worker
│   ├── token-registry.js      # Tier 1 known tokens + registry fetch
│   ├── token-resolver.js      # Three-tier resolution logic
│   ├── price-cache.js         # Price caching layer (60s refresh)
│   └── market-state.js        # AI Pulse state, F&G, dominance (polled from server)
├── content/
│   ├── content-script.js      # Injected into X pages
│   ├── cashtag-detector.js    # Detects $TICKER patterns in DOM
│   ├── popup-injector.js      # Injects hover popup on cashtag click
│   ├── badge-injector.js      # Injects inline mini-badges next to cashtags
│   └── sidebar-injector.js    # Injects sidebar panel
├── popup/
│   ├── popup.html             # Extension icon click popup (quick overview)
│   ├── popup.js
│   └── popup.css
├── sidebar/
│   ├── sidebar.html           # Sidebar panel HTML
│   ├── sidebar.js
│   └── sidebar.css
├── shared/
│   ├── api.js                 # API calls to Spectre server
│   ├── design-tokens.css      # Spectre design system variables
│   └── utils.js               # Shared utilities
├── assets/
│   ├── icons/                 # Extension icons (16, 32, 48, 128)
│   ├── token-icons/           # Cached token logos
│   └── spectre-logo.svg
└── _registry/
    └── known-tokens.json      # Tier 1 token registry (bundled default)
```

Manifest V3

```json
{
  "manifest_version": 3,
  "name": "Spectre AI — Crypto Intelligence for X",
  "version": "0.1.0",
  "description": "AI-powered crypto research overlay for X (Twitter) Smart Cashtags",
  "permissions": [
    "activeTab",
    "storage",
    "alarms"
  ],
  "host_permissions": [
    "https://x.com/*",
    "https://twitter.com/*",
    "https://trade.spectreai.io/*"
  ],
  "background": {
    "service_worker": "background/service-worker.js"
  },
  "content_scripts": [
    {
      "matches": ["https://x.com/*", "https://twitter.com/*"],
      "js": ["content/content-script.js"],
      "css": ["shared/design-tokens.css", "content/content.css"],
      "run_at": "document_idle"
    }
  ],
  "action": {
    "default_popup": "popup/popup.html",
    "default_icon": {
      "16": "assets/icons/icon-16.png",
      "32": "assets/icons/icon-32.png",
      "48": "assets/icons/icon-48.png",
      "128": "assets/icons/icon-128.png"
    }
  },
  "icons": {
    "16": "assets/icons/icon-16.png",
    "48": "assets/icons/icon-48.png",
    "128": "assets/icons/icon-128.png"
  }
}
```

4. DESIGN SYSTEM — MUST MATCH SPECTRE APP

The extension UI must be visually indistinguishable from the main Spectre AI platform. Use the exact same design tokens.

CSS Variables (design-tokens.css)

```css
:root {
  /* Backgrounds */
  --spectre-bg-void: #000000;
  --spectre-bg-base: #0c0c0e;
  --spectre-bg-surface: #131316;
  --spectre-bg-elevated: #1a1a1f;
  --spectre-bg-overlay: #222228;
  --spectre-bg-hover: #2a2a30;
  --spectre-bg-glass: rgba(255, 255, 255, 0.03);
  --spectre-bg-glass-elevated: rgba(255, 255, 255, 0.05);

  /* Text */
  --spectre-text-primary: #ffffff;
  --spectre-text-secondary: rgba(255, 255, 255, 0.72);
  --spectre-text-tertiary: rgba(255, 255, 255, 0.48);
  --spectre-text-muted: rgba(255, 255, 255, 0.32);
  --spectre-text-disabled: rgba(255, 255, 255, 0.16);

  /* Trading Colors */
  --spectre-bull: #10B981;
  --spectre-bull-bright: #34D399;
  --spectre-bull-muted: rgba(16, 185, 129, 0.15);
  --spectre-bull-glow: rgba(16, 185, 129, 0.4);
  --spectre-bear: #EF4444;
  --spectre-bear-bright: #F87171;
  --spectre-bear-muted: rgba(239, 68, 68, 0.15);
  --spectre-bear-glow: rgba(239, 68, 68, 0.4);

  /* Brand Accent */
  --spectre-accent: #8B5CF6;
  --spectre-accent-secondary: #A78BFA;
  --spectre-accent-muted: rgba(139, 92, 246, 0.12);
  --spectre-accent-glow: rgba(139, 92, 246, 0.5);
  --spectre-accent-gradient: linear-gradient(135deg, #8B5CF6 0%, #6366F1 50%, #4F46E5 100%);

  /* Borders */
  --spectre-border-subtle: rgba(255, 255, 255, 0.06);
  --spectre-border-default: rgba(255, 255, 255, 0.08);
  --spectre-border-light: rgba(255, 255, 255, 0.12);

  /* Glass */
  --spectre-glass-bg: rgba(12, 12, 18, 0.95);
  --spectre-glass-blur: blur(40px);
  --spectre-glass-border: rgba(255, 255, 255, 0.08);
```

```
  /* Typography */
  --spectre-font-display: 'Space Grotesk', -apple-system, BlinkMacSystemFont, sans-serif;
  --spectre-font-body: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
  --spectre-font-mono: 'JetBrains Mono', 'SF Mono', 'Fira Code', monospace;

  /* Radius */
  --spectre-radius-sm: 8px;
  --spectre-radius-md: 12px;
  --spectre-radius-lg: 16px;
  --spectre-radius-xl: 20px;
  --spectre-radius-full: 100px;

  /* Motion */
  --spectre-ease-out: cubic-bezier(0.16, 1, 0.3, 1);
  --spectre-duration-fast: 150ms;
  --spectre-duration-normal: 300ms;
  --spectre-duration-slow: 500ms;

  /* Token-specific colors */
  --spectre-btc: #F7931A;
  --spectre-eth: #627EEA;
  --spectre-sol: #14F195;
  --spectre-warning: #F59E0B;
}
```

Component Patterns

Glass Card (popup, sidebar sections):

```
.spectre-glass-card {
  background: var(--spectre-glass-bg);
  backdrop-filter: var(--spectre-glass-blur);
  -webkit-backdrop-filter: var(--spectre-glass-blur);
  border: 1px solid var(--spectre-glass-border);
  border-radius: var(--spectre-radius-lg);
  box-shadow: 0 24px 80px rgba(0, 0, 0, 0.6),
              0 0 1px rgba(255, 255, 255, 0.1),
              inset 0 1px 0 rgba(255, 255, 255, 0.05);
}
```

Edge Highlight (top of popup/cards — 1px gradient line):

```css
.spectre-glass-card::before {
  content: '';
  position: absolute;
  top: 0; left: 0; right: 0;
  height: 1px;
  background: linear-gradient(90deg, transparent 10%, rgba(139, 92, 246, 0.3) 50%, transparent 90%);
}
```

Number Display (prices, F&G, percentages):

```css
.spectre-number {
  font-family: var(--spectre-font-mono);
  font-weight: 500;
  letter-spacing: -0.5px;
}
.spectre-number.hero {
  font-size: 22px;
  font-weight: 300;  /* Ultralight for large numbers — Apple aesthetic */
}
```

Pill Badge (AI Pulse state, sentiment tags):

```css
.spectre-pill {
  display: inline-flex;
  align-items: center;
  gap: 5px;
  padding: 4px 12px;
  border-radius: var(--spectre-radius-full);
  font-size: 11px;
  font-weight: 700;
  letter-spacing: 0.5px;
  text-transform: uppercase;
}
.spectre-pill.risk-off {
  background: var(--spectre-bear-muted);
  border: 1px solid rgba(239, 68, 68, 0.2);
  color: var(--spectre-bear);
}
.spectre-pill.risk-on {
  background: var(--spectre-bull-muted);
  border: 1px solid rgba(16, 185, 129, 0.2);
  color: var(--spectre-bull);
}
```

CTA Buttons:

```css
.spectre-btn-primary {
  background: var(--spectre-accent-gradient);
  color: white;
  border: none;
  border-radius: var(--spectre-radius-sm);
  padding: 10px 16px;
  font-size: 12px;
  font-weight: 600;
  cursor: pointer;
  transition: all var(--spectre-duration-fast) var(--spectre-ease-out);
  box-shadow: 0 4px 14px rgba(139, 92, 246, 0.35);
}
.spectre-btn-primary:hover {
  transform: translateY(-1px);
  box-shadow: 0 6px 20px rgba(139, 92, 246, 0.45);
}
.spectre-btn-secondary {
  background: rgba(255, 255, 255, 0.06);
  color: var(--spectre-text-tertiary);
  border: 1px solid var(--spectre-border-subtle);
  border-radius: var(--spectre-radius-sm);
  padding: 10px 16px;
  font-size: 12px;
  font-weight: 600;
  cursor: pointer;
  transition: all var(--spectre-duration-fast) var(--spectre-ease-out);
}
.spectre-btn-secondary:hover {
  background: rgba(255, 255, 255, 0.1);
  color: var(--spectre-text-primary);
}
```

Golden Rules

1. **All text with `--spectre-` prefix** — Never use raw color values
2. **Numbers always in monospace** — `var(--spectre-font-mono)`
3. **Green = up/gains, Red = down/losses** — Universal trading convention
4. **16px radius on cards**, 8px on buttons/inputs, 100px on pills
5. **1px top gradient edge highlight** on glass cards — this is the Spectre signature
6. **Ultralight (font-weight: 200-300) for hero numbers** — Apple aesthetic for large data
7. **Subtle hover lift** on all interactive elements — `translateY(-2px)`
8. **No emojis in production UI** — Use SVG icons or Unicode symbols only (the concept mockups used emojis for readability; production should not)
9. **Shadow isolation** — Extension CSS must not leak into X's page. Scope all selectors under `.spectre-ext` or use Shadow DOM

---

5. API ENDPOINTS — WHAT EXISTS TODAY

The extension calls the existing Spectre server. **Do not build new backend endpoints.** Use what exists:

Endpoint	Method	Purpose	Used By
`/api/tokens/search?q={query}`	GET	Search tokens by name/symbol	Cashtag resolution (Tier 2)
`/api/tokens/prices?symbols=BTC,ETH,SOL`	GET	Batch price lookup	Sidebar watchlist, popup prices
`/api/tokens/price/:symbol`	GET	Single token price	Popup hero price
`/api/token/details?address={addr}&networkId={id}`	GET	Full token details (name, symbol, socials, market data)	Popup expanded view
`/api/tokens/trending`	GET	Trending tokens	Sidebar trending section

Data the Extension Needs (That We Source From Existing APIs)

Data Point	Source	Refresh
Token price + 24h change	`/api/tokens/prices`	60 seconds
Fear & Greed index	CoinGecko API (already fetched in welcome widget)	5 minutes
BTC Dominance	CoinGecko global data (already fetched)	5 minutes
AI Pulse state	Derived from F&G + price action (same logic as welcome widget)	5 minutes
AI Brief text	`/api/brief/generate` (existing endpoint)	On demand / cached 1 hour
Next economic event	Hardcoded schedule + Finnhub calendar (existing in welcome widget)	1 hour
Trending on X	Count cashtag mentions in visible DOM (client-side)	Real-time

What We Do NOT Have Yet (Do Not Build)

- No real-time WebSocket price feeds in the extension (use polling)
- No user authentication in the extension (anonymous usage, link to spectreai.io for auth)
- No watchlist sync (extension has its own local watchlist in `chrome.storage.local`, no cloud sync)
- No trade execution
- No social sentiment API (we count cashtag mentions client-side as a proxy)

---

6. CASHTAG DETECTION — DOM PARSING ON X

How X Renders Cashtags

On X, cashtags render as `<a>` tags with specific attributes. The extension's content script must detect these.

```javascript
// Detect cashtag elements on X
// X renders cashtags as links with href="/search?q=%24TICKER"
// or with the new Smart Cashtags format (TBD)

const CASHTAG_SELECTORS = [
  'a[href*="/search?q=%24"]',       // Current X cashtag format
  'a[href*="/search?q=$"]',         // Alternative encoding
  '[data-testid="cashtag"]',        // X may add testid for Smart Cashtags
  'a.cashtag',                      // Possible future class
];

function extractTickerFromCashtag(element) {
  // From href: /search?q=%24BTC → BTC
  const href = element.getAttribute('href') || '';
  const match = href.match(/[?&]q=%24([A-Za-z0-9]+)/);
  if (match) return match[1].toUpperCase();

  // From text content: $BTC → BTC
  const text = element.textContent.trim();
  const textMatch = text.match(/^\$([A-Za-z0-9]+)$/);
  if (textMatch) return textMatch[1].toUpperCase();

  return null;
}
```

MutationObserver for Dynamic Content

X loads content dynamically (infinite scroll, new tweets). The content script must watch for new cashtags:

```javascript
const observer = new MutationObserver((mutations) => {
  for (const mutation of mutations) {
    for (const node of mutation.addedNodes) {
      if (node.nodeType === Node.ELEMENT_NODE) {
        const cashtags = node.querySelectorAll(CASHTAG_SELECTORS.join(', '));
        cashtags.forEach(processCashtag);
      }
    }
  }
});

observer.observe(document.body, { childList: true, subtree: true });
```

---

7. WEBMCP FOUNDATION (FUTURE-PROOFING)

Chrome's WebMCP (Web Model Context Protocol) is in early preview. It lets websites expose structured tools that AI browser agents can use. This is NOT for the extension itself — it's for making **spectreai.io** agent-ready.

Why This Matters for Spectre

When AI agents browse the web (via Chrome's built-in AI or third-party agents), they'll be able to:

- Search Spectre's token database
- Get AI Pulse state
- Pull research data for any token
- Get the AI Brief

What to Build Now (on spectreai.io, not the extension)

Add WebMCP declarative tools to Spectre's web pages:

```html
<!-- On spectreai.io token research pages -->
<form data-webmcp-tool="spectre-token-lookup" data-description="Look up crypto token research and AI analysis">
  <input name="symbol" type="text" data-description="Token ticker symbol (e.g., BTC, ETH, SOL, SPECT)" required />
  <input name="chain" type="text" data-description="Blockchain network (ethereum, solana, bsc)" />
</form>

<form data-webmcp-tool="spectre-market-state" data-description="Get current crypto market state including Fear & Greed, A
  <!-- No inputs needed — returns current state -->
</form>
```

This is a future task. For now, just be aware that WebMCP exists and structure Spectre's pages so they can easily add these declarations later. The extension does not use WebMCP — it uses content scripts and the Spectre API directly.

---

8. RESPONSIVE BEHAVIOR

Popup Width

- **Default**: 360px wide (matches X's sidebar proportions)
- **Min**: 320px (small screens)
- **Max**: 400px (large screens)
- Positioned relative to the clicked cashtag, centered horizontally, below the element
- If the popup would overflow the viewport, reposition above the cashtag or shift horizontally

Sidebar Panel

- **Width**: 360px fixed
- Injected as a flex sibling to X's main content column
- X's timeline column should shrink to accommodate (modify X's layout via content script)
- Toggle via extension icon click or keyboard shortcut (Cmd/Ctrl + Shift + S)
- Collapsible — user can close it and it stays closed (persisted in `chrome.storage.local`)

Mobile

- Chrome extensions on mobile are limited. The extension primarily targets desktop Chrome.
- If mobile support is needed later, build a Progressive Web App (PWA) overlay instead.

---

9. SCOPE — WHAT TO BUILD NOW

Phase 1 (MVP — Build This Now)

- [ ] Manifest V3 setup with content script injection on x.com
- [ ] Cashtag detection via DOM parsing + MutationObserver
- [ ] Tier 1 token registry (bundled JSON, top 200 tokens)
- [ ] Tier 2 server search fallback (`/api/tokens/search`)
- [ ] Resolution caching (session + persistent)
- [ ] Hover popup with: token name, price, 24h change, F&G, BTC.D, AI Pulse state
- [ ] "Open in Spectre" deep link button (opens `spectreai.io` with token pre-selected)
- [ ] Extension icon state changes (idle, active on X, market alert)
- [ ] Settings page: enable/disable popup, enable/disable badges, select sidebar position
- [ ] All CSS scoped with Shadow DOM or `.spectre-ext` namespace

Phase 2 (After MVP Works)

- [ ] Sidebar panel with full market overview
- [ ] Inline mini-badges next to cashtags
- [ ] Local watchlist (add tokens from popup, stored in `chrome.storage.local`)
- [ ] AI Brief snippet in popup (fetch from `/api/brief/generate`)
- [ ] Economic event countdown in popup/sidebar
- [ ] Trending cashtags counter (count mentions in visible DOM)

Phase 3 (After Smart Cashtags Launch on X)

- [ ] Smart Cashtag contract address parsing (Tier 3 resolution)
- [ ] Enhanced popup with X's native cashtag data + Spectre intelligence overlay
- [ ] WebMCP tool declarations on spectreai.io
- [ ] User auth sync (login to Spectre via extension, sync watchlist)

---

10. KEY IMPLEMENTATION NOTES

CSS Isolation

The extension's CSS must NEVER affect X's page styling. Two approaches:

Option A — Shadow DOM (Preferred):

```javascript
const host = document.createElement('div');
host.id = 'spectre-ext-root';
const shadow = host.attachShadow({ mode: 'closed' });
shadow.innerHTML = `<style>${spectreCSS}</style><div class="spectre-popup">...</div>`;
document.body.appendChild(host);
```

Option B — Namespaced CSS:

```css
/* Every selector prefixed with .spectre-ext */
.spectre-ext .glass-card { /* ... */ }
.spectre-ext .number { /* ... */ }
```

Shadow DOM is preferred because it guarantees zero CSS leakage in both directions.

Message Passing (Content Script ↔ Background)

```javascript
// Content script → Background: resolve a cashtag
chrome.runtime.sendMessage(
  { type: 'RESOLVE_CASHTAG', ticker: 'BTC' },
  (response) => {
    // response = { name, symbol, price, change24, address, networkId, fgIndex, aiPulse, btcDominance }
  }
);

// Background → Content script: market state changed
chrome.tabs.sendMessage(tabId, {
  type: 'MARKET_STATE_UPDATE',
  data: { aiPulse: 'RISK_OFF', fgIndex: 12, btcDominance: 57 }
});
```

Extension Icon Badge

```javascript
// In background service worker
function updateIconState(marketState) {
  if (marketState.aiPulse === 'RISK_OFF') {
    chrome.action.setBadgeText({ text: '!' });
    chrome.action.setBadgeBackgroundColor({ color: '#EF4444' });
  } else if (marketState.aiPulse === 'RISK_ON') {
    chrome.action.setBadgeText({ text: '↑' });
    chrome.action.setBadgeBackgroundColor({ color: '#10B981' });
  } else if (marketState.eventImminent) {
    chrome.action.setBadgeText({ text: '⏳' });
    chrome.action.setBadgeBackgroundColor({ color: '#F59E0B' });
  } else {
    chrome.action.setBadgeText({ text: '' });
  }
}
```

Deep Linking to Spectre

When "Open in Spectre" is clicked, the extension opens the token research page:

```javascript
// For known tokens with address:
const url = `https://trade.spectreai.io/lite#token/${networkId}/${address}`;

// For symbol-only (fallback):
const url = `https://trade.spectreai.io/lite?search=${symbol}`;

chrome.tabs.create({ url });
```

---

11. TESTING

- Test on x.com with various tweet structures (quote tweets, threads, replies)
- Test cashtag detection with: `$BTC`, `$btc`, `$SPECT`, `$SPECTRE`, unknown tickers
- Test popup positioning at top/bottom/edges of viewport
- Test with X's dark mode and dim mode
- Test resolution speed: Tier 1 should be <5ms, Tier 2 should be <300ms
- Test CSS isolation: ensure zero visual impact on X's native UI
- Test with X's infinite scroll: new tweets should get cashtags detected automatically

---

12. REFERENCE

- **Spectre Design System**: See `DESIGN_SYSTEM.md` and `DESIGN_SYSTEM_ANALYSIS.md` in project docs
- **Spectre API**: See `server/index.js` — all endpoints documented in `API_PLAN.md`
- **Token Registry**: See `KNOWN_TOKEN_ADDRESSES` in `api/codex.js` for existing known tokens
- **Chrome Extension Docs**: https://developer.chrome.com/docs/extensions/
- **Manifest V3 Migration**: https://developer.chrome.com/docs/extensions/develop/migrate
- **WebMCP Early Preview**: https://developer.chrome.com/blog/webmcp-epp
- **X Smart Cashtags**: Feature launching Feb 2026, cashtags will support smart contract addresses